

Error Handling System

ARCHITECTURE & DESIGN DOCUMENT

ARCHITECTURE WORKING FORUM FOR COMMON SERVICES

1. Overview

1.1. Background

XYZ FINTECH's engineers are formed into several Squads which built various in-house developed systems containing modules and services.

Up until today each module and service developed in XYZ FINTECH has its own way to handle errors. Most of them are using hard-coded logic which include decision making based on error code or error description. The decision consists of ABORT, RETRY, or IGNORE.

The above situation has created several issues on the live systems which triggers some amount of complaint ticket raised by the customer and the inconsistent behavior causing the Engineering Team spent lots of working hour to debug and trace.

1.2. Requirement

In order to solve the mentioned situation, it's required to design and then build unified Error Handling Modules with the following capabilities:

- Basic Technical Requirements
 - o It should be designed to serve **ANY** Systems within XYZ FINTECH's
 - o It should be able to receives **SOURCE ERROR CODE** or **SOURCE ERROR DESCRIPTION** as primary input
 - o It should be able to provides **DECISION** and **FORMATTED ERROR CODE**
 - o It should act as **HIGHLY CONFIGURABLE RULE ENGINE**, where all the rules can be configured using **Web CMS** and/or **API**
 - o It should be provided as **SDK** to be used within the application codes and/or **Open API**
- Other Technical Requirements
 - o [SRE] It should be Highly Available
 - o [SRE] It should be Highly Scalable
 - o [OCC] It should record **Transaction ID** for traceability purpose
 - o [ITSEC] It should follow the IT-Security best practices as mentioned in the following document / site: <https://internal.xyz-fintech.com/tech/itsec/security-best-practices>
 - For components or services that are publicly accessible through the internet or exposed to 3rd party should use Secured Transport Layer with minimum TLS 1.2 and token-based authentication
 - For components or services that internally used, it should be implementing Basic Auth at the minimum. This authentication can be represented by the Internal API Gateway
 - The components or services shall be built using latest stable libraries and not continuously updated to comply with the released CVE [Common Vulnerabilities and Exposure] records
 - The components or services shall pass the Penetration Test with NO MAJOR or HIGH RISK record(s)
- Business Requirements
 - o The Modules should be able to handle the following traffic projection

Criteria	2021	2022	2023
----------	------	------	------

Daily Requests	2.5 Mn	4.2 Mn	8.5 Mn
Peak Requests	100 rps	200 rps	500 rps

- Expected Response Time for each requests:
 - Inquiry: 10 secs.
 - Transactions: 20 secs.

2. Technology Stacks

2.1. List of Selected Technology Stacks

These are the selected technology stacks to be utilized on the Error Handling

Name	Type	Version
Golang	Programming Language	1.15.6
EKS	Container Orchestration Platform	1.18
ELB	Load Balancer	2
ElasticSearch	Search Engine	7.9
Memcached	In Memory Cache L2	1.6.9
S3	AWS Object Storage	2006-03-01
go-cache	In Memory Cache L1	2.1.0
go-elasticsearch	Go client for ElasticSearch	7.1.0
aws-sdk-go	AWS SDK for the Go Programming language	1.36.15

2.2. Technology Stacks Usability

These following paragraphs below shall provide information on why it's selected and when or where to use it.

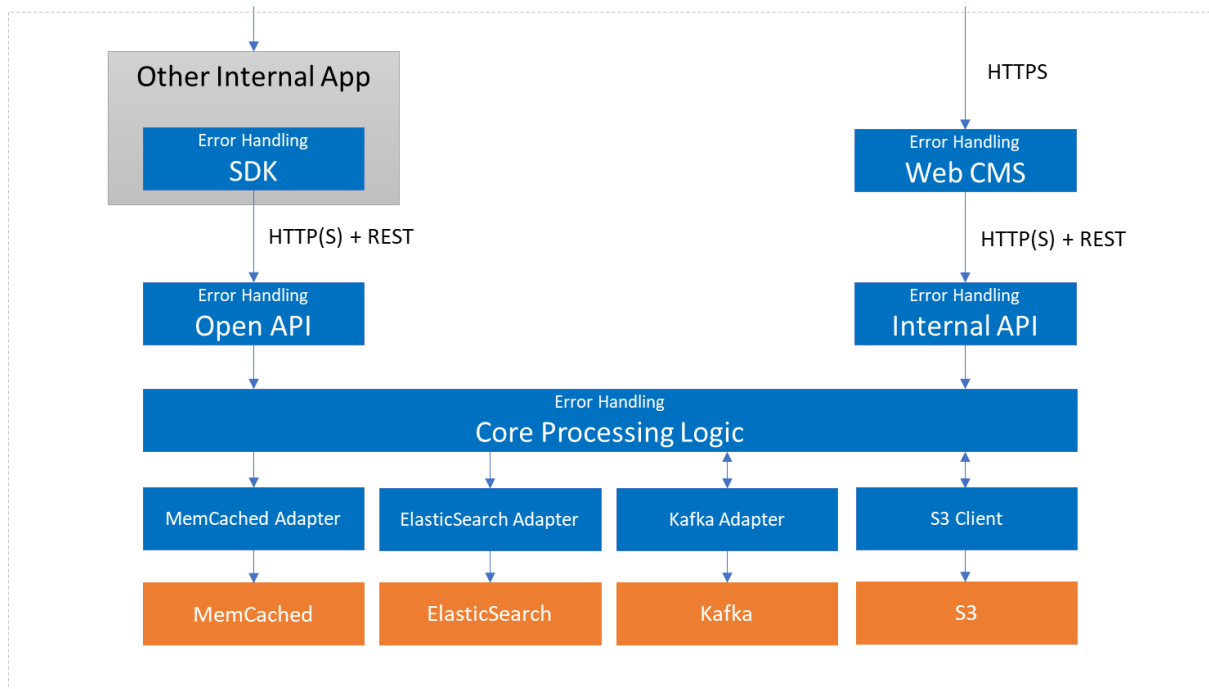
- ElasticSearch:
 - ElasticSearch is selected as **PRIMARY** data store since it support dynamic data model, easy to integrate with as it use REST API, supporting dynamic query, Full Text Search capability, and acceptable performance
 - Within this Error Handling solution, ElasticSearch will be the main “database” to store all error configuration data and error message logs; located right behind the Cache Engine.
- S3:
 - S3 is selected as **SECONDARY** data store since it has 11 of 9 availability [99.999999999%], object storage where it can store any type of file (including json file(s)), provides SDK for various programming language including Go, and it's also serverless which means less hassle on preparing computing resources such as server or VM.
 - Within this Error Handling solution, S3 will be utilized to store static configuration that will be loaded by the Error Handling System upon startup into its Local Cache and act as backup whenever ES is unavailable.
- Go-cache:
 - Go-cache is selected as In-Memory Cached which provides key-value store for application running on a single machine and it's also thread-safe with expiration times
 - Within this Error Handling System, Go-cache will be used as Local Cache, running on the Application Instance :

- **L1 Cache** – as part of the Error Handling SDK
- **L2 Cache** – as part of the Core Processing Application Instance
- Memcached:
 - Memcached is selected for **SERVER-BASED** In-Memory Cache which provides key-value store, segregated from the Application side, and high performance
 - Within this Error Handling System, Memcached will be used as Remote Cache outside of the Application Instance – we can tag it as **L3 Cache**

3. Level 1 - Architecture & Process Flow

2.1. High Level Logical Architecture

Below is the L1 [High Level] Logical Architecture of the Error Handling solution



Components Description

Subject and Code	Description
Error Handling SDK [EH-SDK]	Software Development Kit embedded within Other Internal App to communicate with Error Handling Open API to retrieve Decision and Formatted Error Code
Error Handling Open API [EH-OPA]	API that is exposed to the Error Handling SDK, based on Open API format. This Open API shall not contain any processing logic but input validation for mandatory field(s). Processing Logic obtained by invocation of function within the Error Handling Core Processing Logic component

Error Handling Web CMS [EH-WCMS]	Web Application that being used by Configuration Management PIC to manage Error Handling data
Error Handling Internal API [EH-INA]	Management API that is only exposed to Error Handling Web CMS This Internal API shall not contain any processing logic but input validation for mandatory field(s). Processing Logic obtained by invocation of function within the Error Handling Core Processing Logic component
Error Handling Core Processing Logic [EH-CPL]	Brain for the Error Handling Sub-System which orchestrate processing logic and invocation of external components for data store and messaging through the Application Adapter(s)
Memcached Adapter [MCH-ADP]	Common Component within XYZ FINTECH that wrap communication to Memcached Server(s)
ElasticSearch Adapter [ELS-ADP]	Common Component within XYZ FINTECH that wrap communication to ElasticSearch Server(s)
Kafka Adapter [KFK-ADP]	Common Component within XYZ FINTECH that wrap communication to Kafka Broker Server(s)
S3 Client [S3-CLI]	Wrapper component for AWS S3 SDK Go

3.2. High Level Process Flow in L1 Architecture

Basic rule of thumb of request handling shall be implemented as follow:

- **READ** operation will be processed **synchronously**
- **CREATE, UPDATE** and **DELETE** will be processed in hybrid mode: synchronous and asynchronous

Below is the how the process flow within L1 Architecture

[*] Within the Process Flow, component name can be substituted with component-code

- [Error Handling Configuration Update Process Flow](#)

Step #	Description
1	CMS Operator perform changes on EH-WCMS and click submit button. EH-WCMS send the request to the EH-INA
2	EH-INA receive the request, then perform input validation. If input valid, EH-INA will invoke function within EH-CPL on Step 3 If input invalid, EH-INA will immediately respond with HTTP 400 Bad Request
3	EH-CPL will attempt to store data by using interface on ELS-ADP . Detail is on Step 4 If storing data OK , EH-CPL will produce event to Kafka through KFK-ADP to notify the topic's consumers for data changes; and response back to EH-INA with HTTP 201 If process FAILED , EH-CPL will response back with formatted Error Code to the EH-INA
4	ELS-ADP transform input parameters into required <i>ElasticSearch</i> message and query format and then send to <i>ElasticSearch</i> This is synchronous call, thus result will be provided back to EH-CPL on Step 3
5	KFK-ADP transform input parameters into appropriate message document and send it through it Producer to specified topic in <i>Kafka</i>

6	Associated Kafka Consumer Group for Memcached receive event from Step 5 and call the interface in MCH-ADP to update the cache
7	Associated Kafka Consumer Group for S3 receive event from Step 5 and call the interface in S3-CLI to store the update in specified bucket and location
8	Associated Kafka Consumer Group for L2 Cache on Open API instance receive event from Step 5 and call its internal function to update its local cache (L2 Cache)

- Error Handling Request Processing Flow

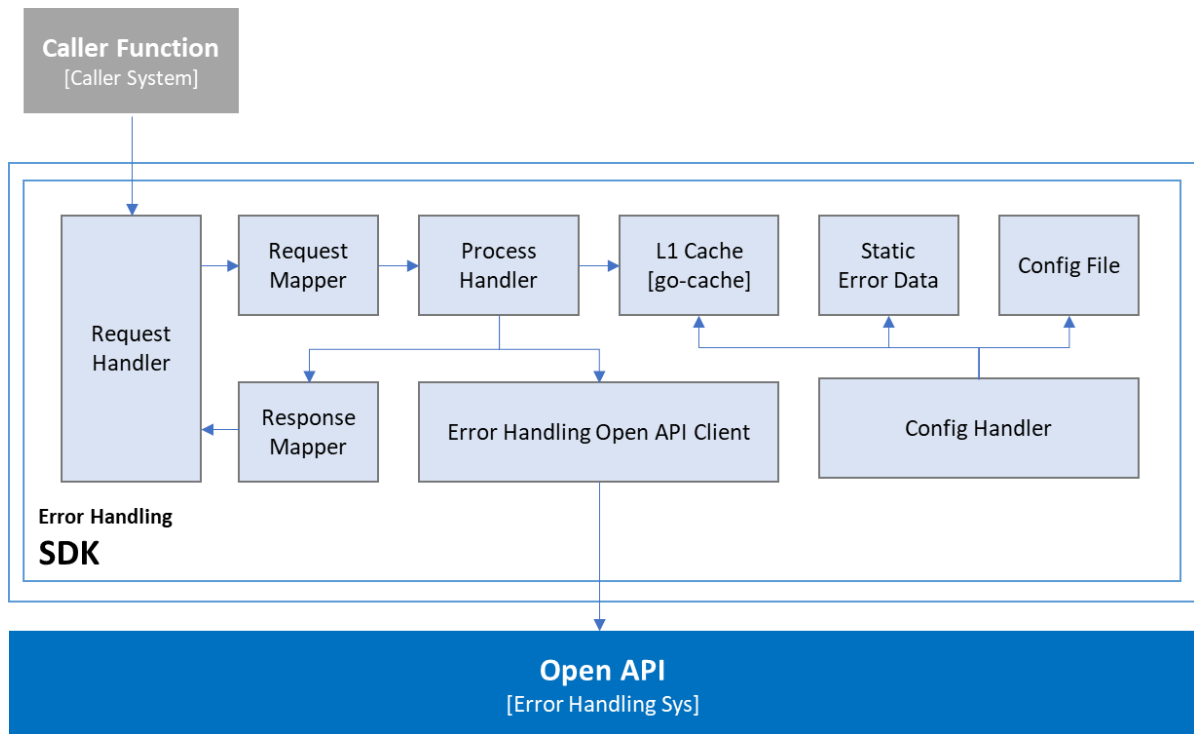
Step #	Description
1	An error occurred on Caller System triggered the use of Error Handling function in EH-SDK . The function within EH-SDK will validate the input. If input VALID , it will continue the process on Step 2 If input INVALID , it will terminate and response back with HTTP 400 Bad Request
2	EH-SDK check its internal cache [L1 Cache]. if data FOUND , it will execute Step 3 if data NOT FOUND , it will terminate and provide the result from L1 Cache to the caller function
3	EH-SDK send the request to EH-OPA
4	EH-OPA receive the request then perform input validation. If input valid, EH-OPA will invoke function within EH-CPL on Step 5 If input invalid, EH-OPA will immediately respond with HTTP 400 Bad Request
5	EH-OPA perform check on its local cache [L2-Cache]. If data FOUND in L2 Cache, it will immediately return the result. If data NOT FOUND in L2 cache, it continues to execute Step 6
6	EH-CPL perform check on Memcached data [L3 Cache] using MCH-ADP . If data FOUND in L3 Cache, it will immediately return the result. If data NOT FOUND in L3 Cache, it continues to execute Step 7
7	EH-CPL perform check on ElasticSearch using ELS-ADP with hierarchical mechanism. Detail about hierarchical search handling will be explained in L1 Architecture section. If data FOUND in ElasticSearch, it will return the result. If data NOT FOUND in ElasticSearch, it will respond with default error handle response message for Data Not Found . In the case of ElasticSearch service is Not Available, goes to Step 8
8	EH-CPL attempt to retrieve data using S3-CLI from Object Storage (S3) If data FOUND in S3, it will return the result. If data NOT FOUND in S3, it will respond with default error handle response message for Data Not Found .

4. Level 2 - Architecture & Process Flow

4.1. Error Handling SDK Logical Architecture & Process Flow

- Error Handling SDK L2 Architecture Diagram

This architecture diagram below with arrow lines illustrate flows within Error Handling SDK



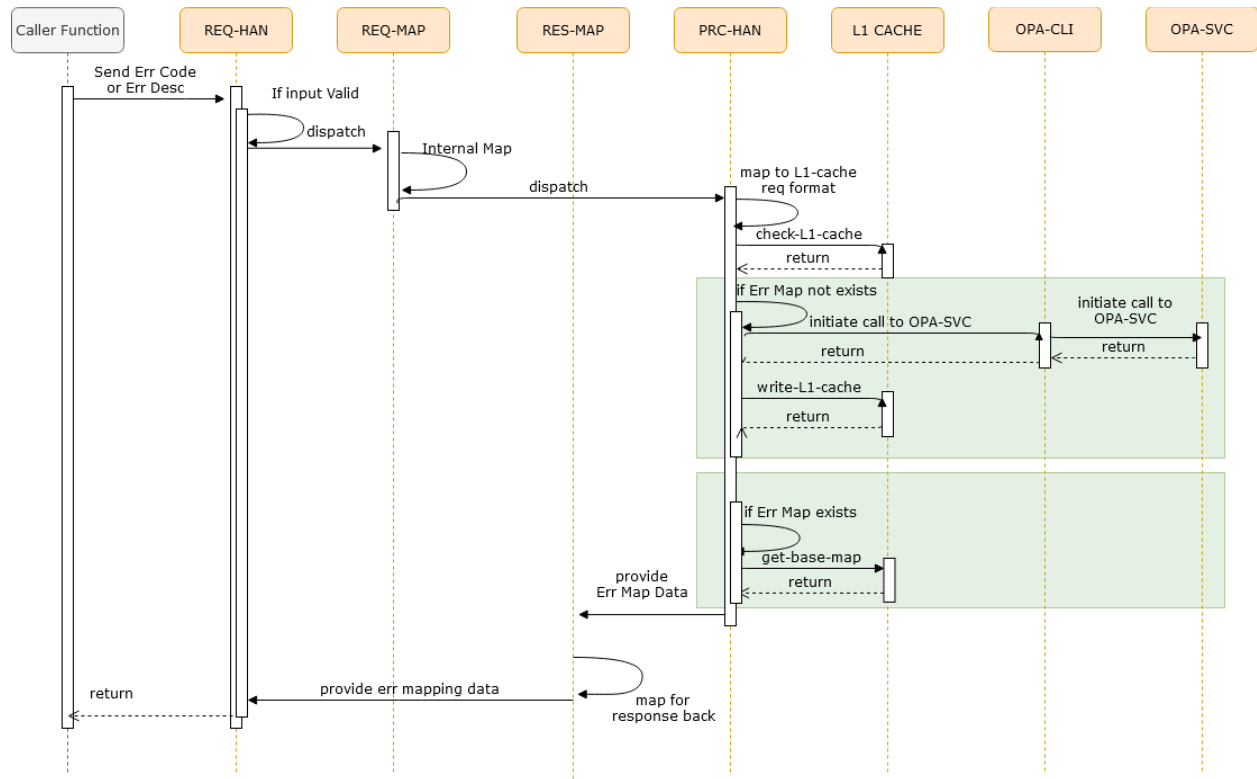
- Error Handling SDK Components Description

Subject	Description
Request Handler [REQ-HAN]	Sets of functions that being called by the Caller Function to obtain Formatted Error Code and Action Points
Request Mapper [REQ-MAP]	Map Input Parameters from Request Handler to be passed to Process Handler
Response Mapper [RES-MAP]	Map Output values that is provided by Process Handler into specific functions output
Process Handler [PRC-HAN]	“Right-Side Brain” of the SDK, which orchestrate the request processing into several components. This component also perform decision on cache retrieval and store vs sending request to Open API
Error Handling Open API Client [OPA-CLI]	Connector to the Error Handling System’s Open API Server
L1 Cache	Level-1 Cache for this SDK, containing : - Static Error Data - Configurations such as Address and credentials required for Open API - Portion of Error Data obtained from Error Handling Open API
Static Error Data [STC-ERR]	JSON based file containing basic error data mapping such as Response Code and Actions for “Connection timed out”, “Bad Request” due to invalid input
Config File [CFG-FILE]	Properties file containing data such as Address and credentials required for Open API

Config Handler [CFG-HAN]	Configuration Handler which will be initiated during the initial loading of the Error Handling SDK. This component will load the Static Error Data and Config File into L1 Cache
-------------------------------------	---

- Error Handling SDK Message Process

Here's the sequence diagram for Error Handling SDK



Here's the Step-by-Step process in table format for Error Handling SDK

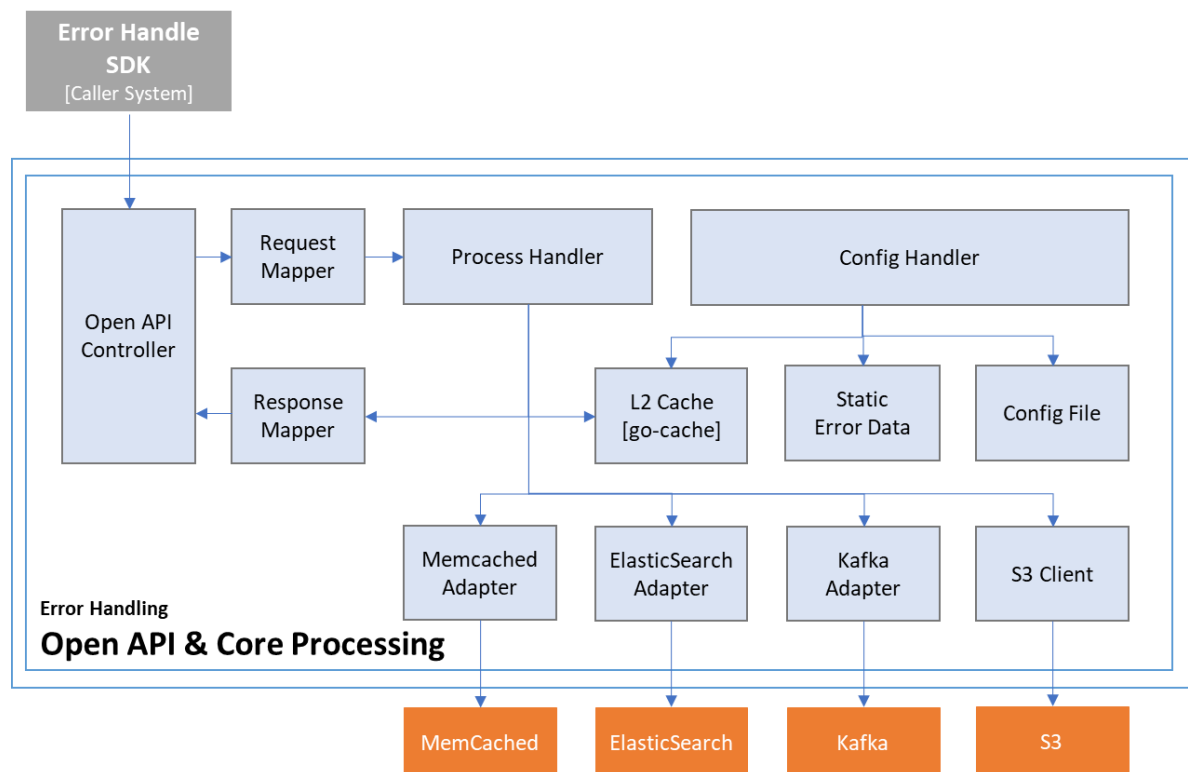
Step #	Description
1	REQ-HAN receive Error Code or Error Response from Caller Function. If input VALID , it will continue to Step 2 If input INVALID , it will response with "Bad Request"
2	REQ-MAP format received input to internal data model
3	PRC-HAN receive formatted message from REQ-MAP then convert the data into L1 Cache compliant format.
4	PRC-HAN check the L1 Cache data using formatted key If the Error Mapping Data response from L1 Cache is NULL then PRC-HAN will initiate to call Open API interface on Step 5 If the Error Mapping Data response from L1 Cache is NOT NULL then PRC-HAN will send it to Response Mapper on Step 8
5	PRC-HAN call the Open API Interface using OPA-CLI to get Error Mapping Data. [*] Address and credential for OPA-CLI has been set during Application startup
6	PRC-HAN update L1 Cache Data based on response on previous step

7	PRC-HAN pass the Error Mapping Data to RES-MAP
8	RES-MAP maps the Error Mapping Data into required format
9	REQ-HAN provide the data back to Caller Function

4.2. Open API & Core Processing Logical Architecture & Process Flow

- Error Handling Open API & Core Processing L2 Architecture Diagram

Below is architecture diagram with arrow lines to illustrate flows within Error Handling SDK



- Open API & Core Processing Components Description

Subject	Description
Open API Controller [OPA-CTL]	Sets of API that being called by the Error Handler SDK to obtain Formatted Error Code and Action Points
Request Mapper [REQ-MAP]	Map Input Parameters from Request Handler to be passed to Process Handler
Response Mapper [RES-MAP]	Map Output values that is provided by Process Handler into specific functions output
Process Handler [PRC-HAN]	“Right-Side Brain” of the Core Processing, which : - Orchestrate the request processing into several components including Adapters. - Perform decision on cache retrieval and store vs sending request to Memcached or ElasticSearch or S3 based on priorities and availability of the target system
Memcached Adapter [MCH-ADP]	Connector to the Memcached server

ElasticSearch Adapter [ELS-ADP]	Connector to the ElasticSearch service
Kafka Adapter [KFK-ADP]	Connector for the Kafka Messaging Broker
S3 Client [S3-CLI]	Client component for S3
L2 Cache	Level-2 Cache for this SDK, containing : - Static Error Data - Configurations for Adapters - Portion of Error Data obtained from Data Store (Memcached, Elasticsearch or S3)
Static Error Data [STC-ERR]	JSON based file containing basic error data mapping such as Response Code and Actions for “Connection timed out”, “Bad Request” due to invalid input
Config File [CFG-FILE]	Properties file containing data such as Address, credentials, timeout settings, health-check intervals, etc required for Adapters
Config Handler [CFG-HAN]	Configuration Handler which will be initiated during the initial loading of the Open API and Core Processing. This component will load the Static Error Data and Config File into L2 Cache

- [Open API & Core Processing Message Process](#)

[*] Configs for Adapter has been pre-loaded during Application startup sequence to avoid getting configs every time Adapter being invoked

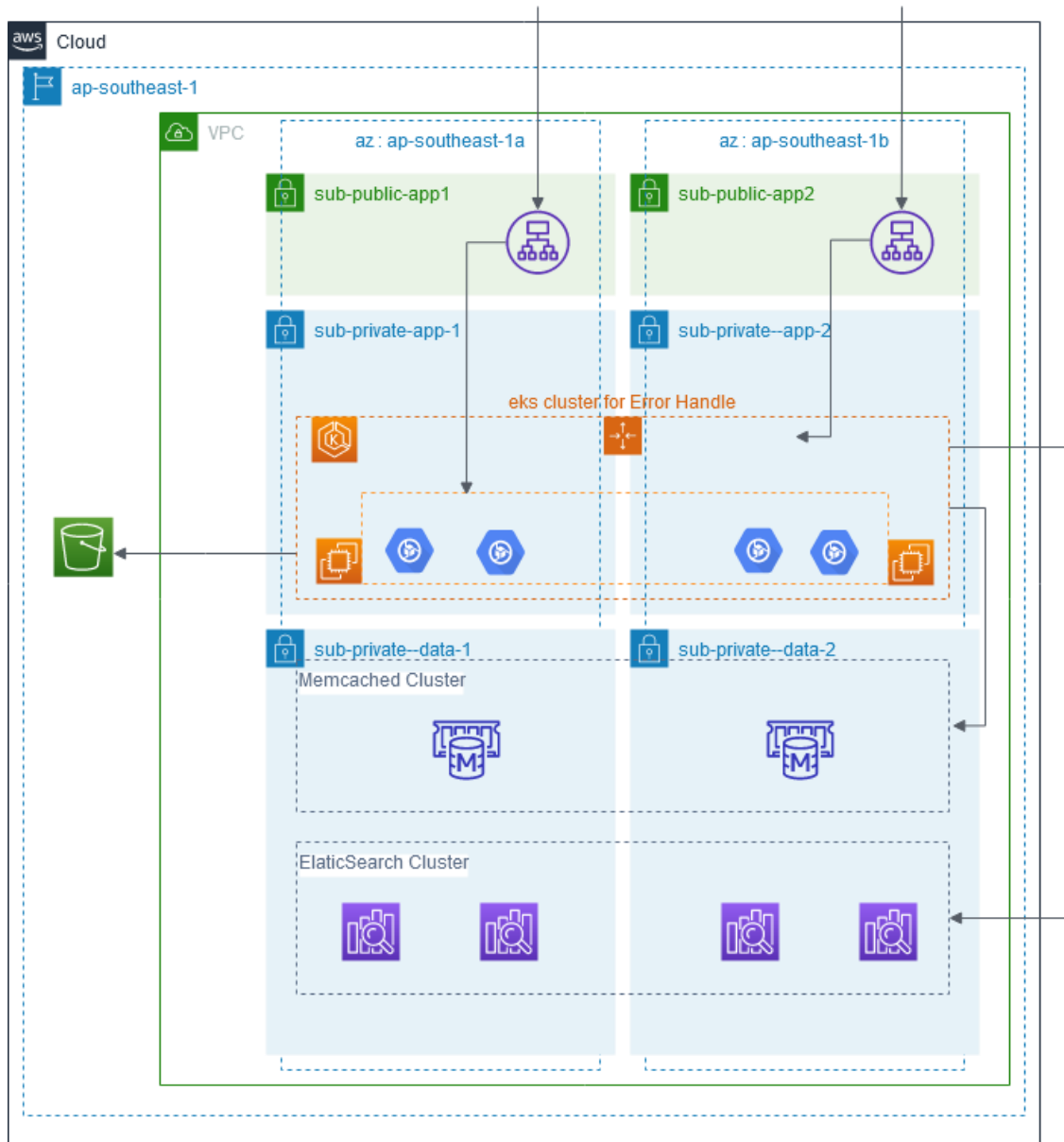
Step #	Description
1	OPA-CTL receive Error Code or Error Response from EH-SDK and validate If input VALID , it will continue to Step 2 If input INVALID , it will response with “Bad Request”
2	REQ-MAP format received input to internal data model and call the PRC-HAN module
3	PRC-HAN receive data from REQ-MAP and convert the data into L2 Cache compliant format.
4	PRC-HAN check the L2 Cache data using formatted key If the Error Mapping Data response from L2 Cache is NULL then PRC-HAN will initiate to call check to Memcached (L3 Cache) interface on Step 5 If the Error Mapping Data response from L2 Cache is NOT NULL then PRC-HAN will send it to Response Mapper on Step10
5	PRC-HAN check the Memcached [L3 Cache] data through MCH-ADP using formatted key If the Error Mapping Data response from L3 Cache is NULL then PRC-HAN will initiate to call check to ElasticSearch service on Step 6 If the Error Mapping Data response from L2 Cache is NOT NULL then PRC-HAN will send it to Response Mapper on Step 9
6	PRC-HAN call the function within ELS-ADP to query the Error Mapping Data with tiered search If the Error Mapping Data response from ElasticSearch Cache is NULL then PRC-HAN will mark this particular Error Code or Error Description as unknown and goes to Step 7

	If the Error Mapping Data response from L2 Cache is NOT NULL then PRC-HAN will send it to Response Mapper on Step 9
7	PRC-HAN retrieve Error Mapping Data for <i>Unknown Error</i> on L2 Cache data using formatted key, then goes to Step 9
8	PRC-HAN publish the newly found data from ElasticSearch to Kafka topic using KFK-ADP
9	PRC-HAN publish the Input Parameters and Response Values to Kafka topic using KFK-ADP for logging purpose.
9	PRC-HAN pass the result to RES-MAP
10	RES-MAP format the Error Mapping Data into required external format
11	RES-MAP pass the formatted Error Mapping Data to OPA-CTL
12	OPA-CTL provides the response back to Caller Function

5. Deployment Architecture on Cloud

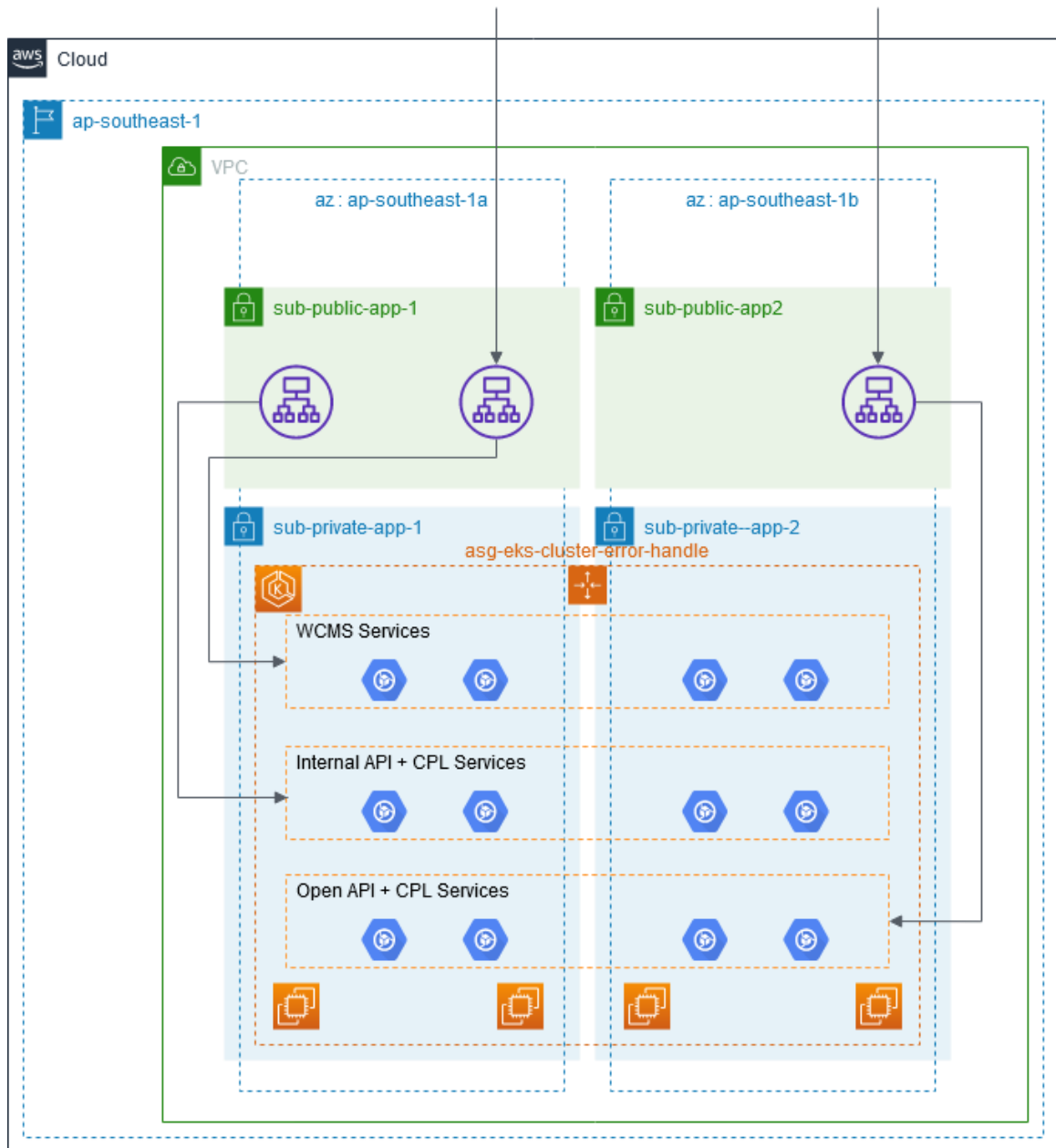
5.1. Cloud Architecture for Error Handling System

Below is the cloud architecture for Error Handling System



5.2. Container Architecture for Error Handling System

Below is the cloud architecture for Container implementation of Error Handling System



6. Data Model

6.1. Data Model for Error Handling Message

Field naming is using camel case model, started with lower case.

- Error Handling Request Message

Field Name	M/O/C	Field Type	Field Description
trxId	M	varchar[50]	Transaction ID. This is for logging and tracking purpose

callerSysName	M	varchar[50]	System Name that initiate the Error Handling Open API call. Value should be all in lower case. e.g : phoenix
callerClassName	M	varchar[50]	Class Name that initiate the Error Handling Open API call e.g : AccountDao
callerFuncName	M	varchar[50]	Function Name that initiate the Error Handling Open API call e.g : InsertAccount
srcErrSysName	M	varchar[50]	System name which originally throws the error. Value should be all in lower case e.g : postgresql
srcErrCode	C	varchar[20]	Error Code that throwed by the Source System. Within the request payload, at least one of the srcErrCode or srcErrDesc should be present. e.g : 22026, ORA-00902
srcErrDesc	C	varchar[255]	Error Code throwed by the Source System. Within the request payload, at least one of the srcErrCode or srcErrDesc should be present. e.g : 'Request timed out' , 'string_data_length_mismatch'
retryCount	M	int	Number of retry count. For 1 st error, retryCount shall be filled with 0 since no retry initiated yet.

- Error Handling Response Message

Field Name	M/O/C	Field Type	Field Description
fmtErrCode	M	varchar[50]	Formatted Error Code
actType	M	varchar[10]	Action Type. AF = Abort and Finish. Caller should abort the process without any rollback sequence and throws back to the upper Function or return to the original requestor (e.g : Front End) AR = Abort and Rollback. The current execution should be aborted but the Caller is mandatory to initiate Rollback Sequence RE = Retry execution

			IG = Ignore. Caller can skip the current error and continue with subsequent step if there's any
maxRetryCount	C	int	Number of max retry that's allowed to be executed by the Caller.

6.2. Data Model for ElasticSearch

With current version of ElasticSearch is not allowing Type Mapping anymore, each document type will have different index

- Data Model for conf_error ElasticSearch index

Field Name	M/O/C	Field Type	Field Description
caller_sys_name	M	varchar[50]	System Name that initiate the Error Handling Open API call. Value should be all in lower case. Use 'default' as default value
src_err_sys_Name	M	varchar[50]	System name which originally throws the error. Value should be all in lower case
src_err_code	M	varchar[20]	Error Code that throwed by the Source System. Within the request payload, at least one of the srcErrCode or srcErrDesc should be present.
src_err_desc	M	varchar[255]	Error Code throwed by the Source System. Within the request payload, at least one of the srcErrCode or srcErrDesc should be present.
fmt_err_code	M	varchar[50]	Formatted Error Code
act_type	M	varchar[10]	Action Type. AF = Abort and Finish. Caller should abort the process without any rollback sequence and throws back to the upper Function or return to the original requestor (e.g : Front End) AR = Abort and Rollback. The current execution should be aborted but the Caller is mandatory to initiate Rollback Sequence RE = Retry execution IG = Ignore. Caller can skip the current error and continue with subsequent step if there's any
nax_retry_count	M	int	Number of retry count.

			For 1 st error, retryCount shall be filled with 0 since no retry initiated yet.
conf_status	M	int	1 = Active 0 = Inactive -1 = To be deleted

- Data Model for conf_system ElasticSearch index

Field Name	M/O/C	Field Type	Field Description
sys_name	M	varchar[50]	System Name
sys_hash_code	M	varchar[50]	System Hash Code that generated every time there's changes initiated from Web CMS.
sys_hash_time	M	varchar[20]	Timestamp of latest modification date of data in the conf_error for this particular sys_name Format : YYYYMMDD HH24:MI:ss.ZZZ
sys_hash_status	M	varchar[20]	Either empty space or 'in_progress_update'

7. Other Technical Consideration

7.1. System Performance Estimation

- Assumptions for System Performance

The following table consists of parameters that may impact performance calculation

Item	Value	Description
L1 Cache response	1 ms	Average time for cache in EH-SDK to response
L2 Cache response	1 ms	Average time for cache in EH-CPL to response
L3 Cache response	10 ms	Average time for Memcached to response
ElasticSearch response	100 ms	Average time for Elasticsearch to response query
Timeout Configuration	5 secs	Timeout setting for Adapter
Pack-Unpack or Serialize-Deserialize Object	1 ms	Average time for Request Mapper and Response Mapper
Network Latency	1 ms	
Internal Component Response [*]	1 ms	

- Calculated System Performance

The following table consists of performance calculation on different scenarios

Scenario	Value	Description
Error Mapping Data exists in EH-SDK	7 ms	- Serialize/Deserialize and Mapping on REQ-MAP and RES-MAP [2x] - In Transit PRO-HAN (2x) - L1 Cache Response
Error Mapping Data not exists in EH-SDK and OPA-API timed out	5.01 s	- Serialize/Deserialize and Mapping on REQ-MAP and RES-MAP [4x] - In Transit PRO-HAN (3x) - L1 Cache Response (2x) - L2 Cache Timeout [1x] - Network Latency [1x]
Error Mapping Data exists in OPA-API's L2 Cache	26 ms	- Serialize/Deserialize and Mapping on REQ-MAP and RES-MAP [4x] - In Transit PRO-HAN (4x) - L1 Cache Response (2x) - L2 Cache Response [1x] - Network Latency (2x)
Error Mapping Data exists in Memcached	100 ms	- Serialize/Deserialize and Mapping on REQ-MAP and RES-MAP [4x] - In Transit PRO-HAN (5x) - L1 Cache Response (2x) - L2 Cache Response [1x] - Network Latency (2x)

7.2. System Size Estimation

- Assumptions for System Size

The following table consists of parameters that may impact system size calculation

Item	Value	Description
Item Data Size for Error Mapping and System Config	1 KB	Size per Item/row
L1 Cache Static Data	100 rows	
Identified System Wide Error Mapping Data	2.000 rows	
Daily Call for Error Mapping	10K	
Average System Config Records Per System	5 rows	
Error Handling SDK	10 MB	
Error Handling Application	100 MB	

- Calculated System Size

The following table consists of size calculation on different scenarios

Item	Value	Description
Compute Node for Error Handling k8s	4	AWS EC2 m5.xlarge @ 4vCPU and 16 GB RAM
Memcached Node	2	AWS cache.m5.large @ 2vCPU and 6.38 GB RAM
ElasticSearch Node	4	AWS m5.large @ 2vCPU and 8 GB RAM
S3 for Logs	3.65 GB	10K daily * 1 KB * 365

8. Referenced Information

- AWS EC2 Instance Type
<https://aws.amazon.com/ec2/instance-types/>
- AWS ElasticSearch Pricing
<https://aws.amazon.com/elasticsearch-service/pricing/?nc=sn&loc=3>
- Amazon ElastiCache Supported Node Types
<https://docs.aws.amazon.com/AmazonElastiCache/latest/mem-ug/CacheNodes.SupportedTypes.html>